Customer Churn Prediction

End-to-End Data Science Capstone: Project Documentation

Author	Sumit Gupta
Program	Data Science, The Developers Arena (Week 12 Capstone)
GitHub repository	https://github.com/SumitGupta9627/customer-churn-capstone
Dataset	customer_churn.csv (500 customers, 9 columns)
Tools	Python, pandas, scikit-learn, Matplotlib, Seaborn, Flask, Git

Contents: 1 Project overview and objectives | 2 Setup and installation | 3 Code structure | 4 Screenshots of the working application | 5 How the technical requirements were met | 6 Methodology and results | 7 Business impact and recommendations | 8 Testing evidence | 9 Limitations

1. Project overview and objectives

Business problem. A subscription company loses customers every month. Keeping a customer costs far less than winning a new one, so the company wants to find customers who are about to leave and contact them first.

Objectives

- Explore the data and find what drives churn.
- Build a model that predicts which customers will churn (target: ROC-AUC 0.90 or higher and recall 80% or higher).
- Turn the model into a working web application and prediction API.
- Estimate the money a retention campaign could save and give clear business recommendations.

Data. 500 customers, one row each, with tenure, monthly and total charges, contract type, payment method, paperless billing, senior-citizen flag and the target *Churn* (53 of 500, or 10.6%, churned). Quality checks found no missing values, duplicates or negative numbers.

Key findings

- All churn happens in the first 12 months: 53 of 84 customers (63%) churned in that period, while none of the 416 customers who passed month 12 did.
- Month-to-month contracts churn at 20.6%, against 4.3% (one year) and 6.9% (two year).
- New customers paying more than 100 a month churn at 79%, against 38% on cheaper plans.

2. Setup and installation instructions

Requires **Python 3.11 or newer**. Run all commands from the project folder (the folder that contains `README.md`).

```
git clone https://github.com/SumitGupta9627/customer-churn-capstone.git
cd customer-churn-capstone

python -m venv .venv
.venv\Scripts\activate          (Windows)
source .venv/bin/activate     (Mac / Linux)

pip install -r requirements.txt
python -m src.train                          # optional: retrain the model
python -m unittest discover -s tests         # run the 16 tests
python deployment/app.py                     # start the web app
```

Then open `http://127.0.0.1:5000` in a browser. To read the notebook run `pip install jupyter` and `jupyter notebook capstone_project.ipynb` from the project folder.

3. Code structure

Path	Purpose
README.md	Project overview, setup, API usage, testing evidence
capstone_project.ipynb	Main analysis: EDA, modeling, evaluation, business impact (already executed)
src/config.py	Paths, column names and constants in one place
src/data_loader.py	Loads the data and runs quality checks
src/features.py	Feature engineering and the preprocessing pipeline
src/train.py	Compares three models, tunes the best, saves the model and plots
src/evaluate.py	Metrics, confusion matrix, ROC curve, feature importance
src/predict.py	Input validation and prediction used by the app
src/business.py	Retention-campaign money estimate
data/	customer_churn.csv and data dictionary
reports/	Technical documentation, business report, metrics, figures, screenshots, test log
deployment/	Flask app, web page template, trained model, Dockerfile
presentation/	13-slide business presentation (PowerPoint)
tests/	16 automated tests

4. Screenshots of the working application

The Flask app has a form for customer details and returns the churn probability, a risk level and a suggested action.

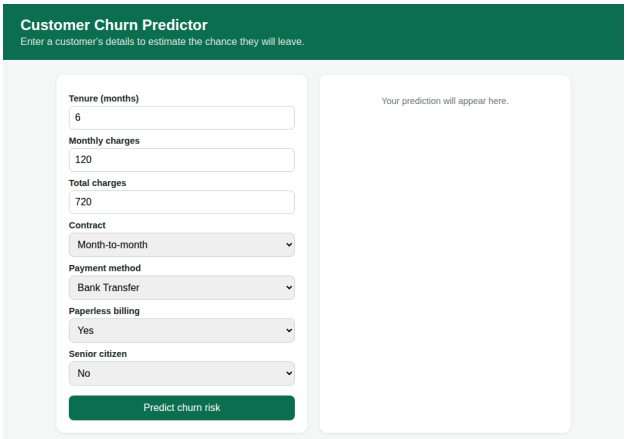


Figure 1. Home page with the input form

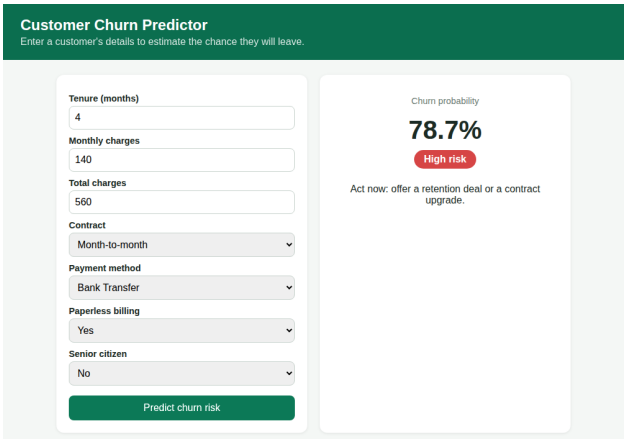


Figure 2. High-risk customer: new, month-to-month, high bill (78.7%)

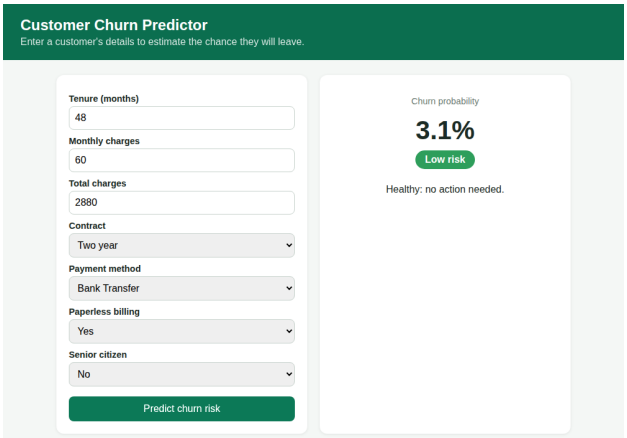


Figure 3. Low-risk customer: long tenure, two-year contract

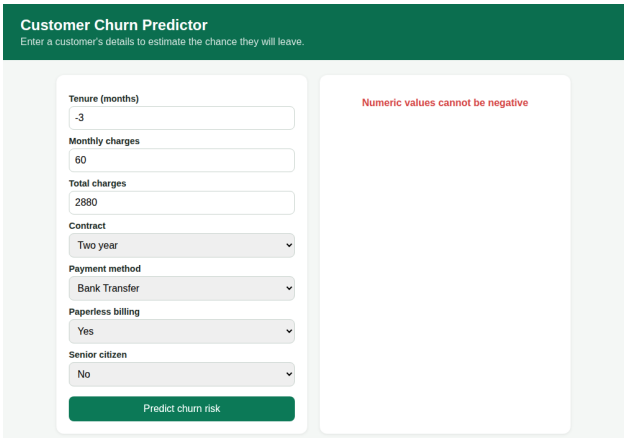


Figure 4. Input validation: a negative value is rejected with a clear message

API example (POST /predict)

```
curl -X POST http://127.0.0.1:5000/predict -H "Content-Type: application/json" \
  -d '{"Tenure":4,"MonthlyCharges":140,"TotalCharges":560,"SeniorCitizen":0,
       "Contract":"Month-to-month","PaymentMethod":"Electronic Check","PaperlessBilling":"Yes"}'

Response: {"churn_probability": 0.8372, "prediction": 1, "risk_level": "High"}
```

5. How the technical requirements were met

Requirement	How it was met	Where to find it
Complete end-to-end workflow	Problem definition, data quality check, EDA, feature engineering, model comparison, tuning, evaluation, deployment and business recommendations	capstone_project.ipynb, src/
Professional documentation and reporting	README, technical documentation, business report, data dictionary and this document	README.md, reports/
GitHub repository with organized structure	Required folders and files, modular code, requirements.txt, .gitignore	Repository root

Requirement	How it was met	Where to find it
Business presentation with actionable insights	13 slides from problem to recommendations, with charts and speaker notes	presentation/
Basic model deployment demonstration	Flask web app and JSON API with validation; Dockerfile for production	deployment/
Testing evidence	16 automated tests covering data, model, API and business logic (all pass)	tests/, reports/test_results.txt

6. Methodology and results

Step	What was done
Data quality	Checked columns, missing values, duplicates, duplicate IDs and negative numbers; all passed
EDA	Churn balance, churn by contract, payment, billing and tenure, box plots, correlation matrix
Features	AvgMonthlySpend, IsNewCustomer (tenure 12 or less), HighCharges (monthly charges above 100)
Preprocessing	StandardScaler for numbers and OneHotEncoder for categories, inside one scikit-learn Pipeline
Split	80% train (400) and 20% test (100), stratified, random_state 42
Model comparison	5-fold cross-validation, ROC-AUC, on training data only
Tuning	Grid search over 27 Random Forest settings; class imbalance handled with balanced class weights
Evaluation	One final check on the untouched test set

Model comparison (5-fold cross-validation ROC-AUC)

Model	Mean AUC	Std
Logistic Regression	0.9922	0.0046
Random Forest	0.9928	0.0038
Gradient Boosting	0.9940	0.0060

All three models score about 0.99, so the churn signal is strong. A Random Forest was chosen because it is robust and easy to explain, then tuned (best settings: max_depth 3, min_samples_leaf 5, 300 trees; cross-validated AUC 0.993).

Final results on the unseen test set (100 customers, 11 churners)

Accuracy	Precision	Recall	F1	ROC-AUC
0.930	0.611	1.000	0.759	0.997

Confusion matrix: 82 correct stays, 11 churners caught, 7 false alarms, 0 churners missed. Recall of 1.00 means no churner was missed; precision of 0.61 means about 4 in 10 flagged customers would have stayed anyway, which is acceptable because a retention offer is cheap compared with losing a customer.

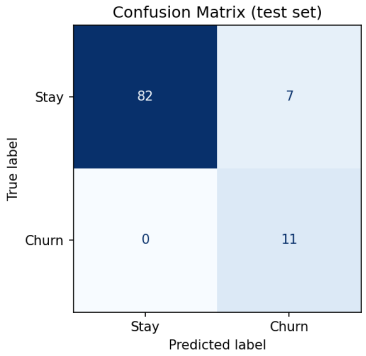


Figure 5. Confusion matrix

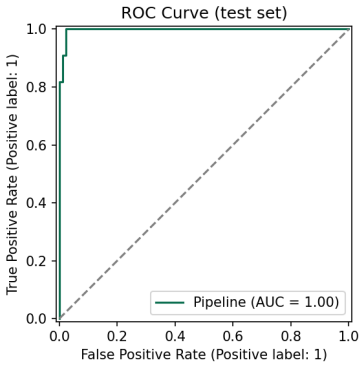


Figure 6. ROC curve

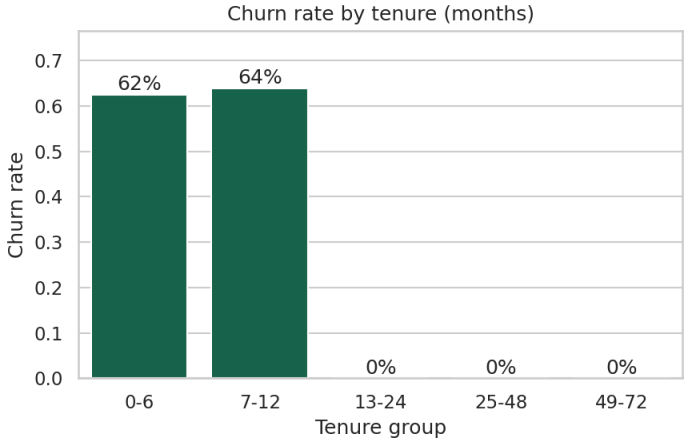


Figure 7. Churn rate by tenure: all churn happens in the first 12 months

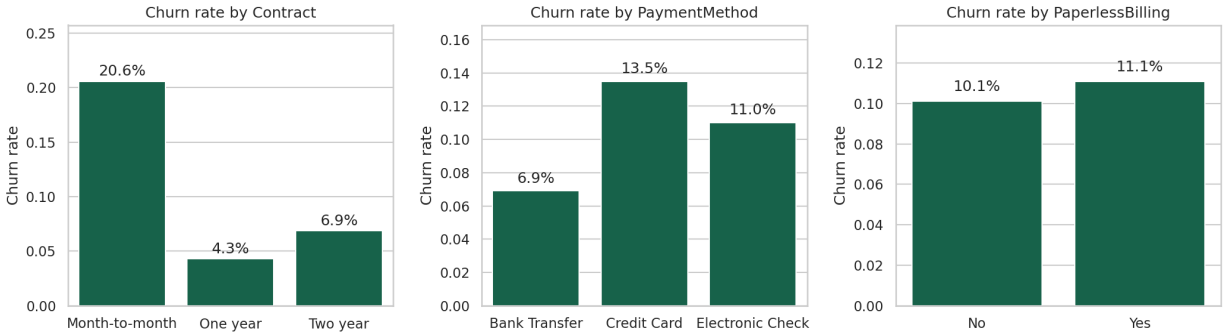


Figure 8. Churn rate by contract, payment method and paperless billing

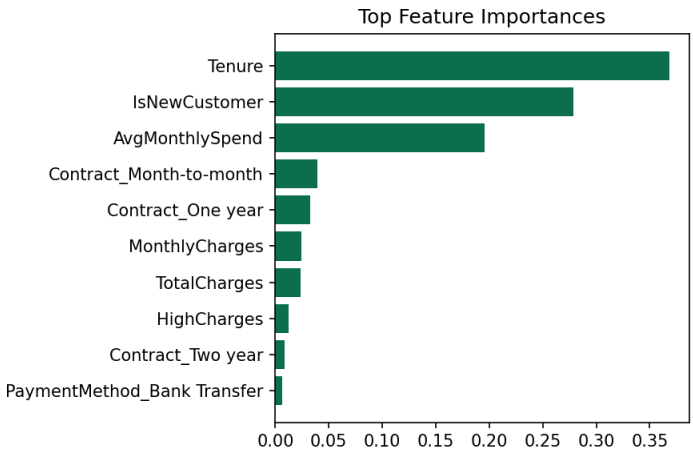


Figure 9. Top feature importances: tenure and the new-customer flag dominate, matching the EDA

7. Business impact and recommendations

Every customer was scored with out-of-fold predictions (each predicted by a model that never saw them). The company's choices were compared: do nothing, or contact every customer the model flags.

Item	Value
Customers flagged / churners reached	81 flagged, 53 of 53 churners reached
Annual revenue carried by churned customers	82,536
Campaign cost	11,444
Revenue saved	24,761
Net benefit	**13,316**

Assumptions: an offer costs 10% of the customer's annual revenue, and 30% of contacted churners accept it and stay. The campaign breaks even at a save rate of about 14%. Amounts are in the dataset's currency and are estimates.

Recommendations

- **Focus on months 1 to 12.** Add onboarding calls and check-ins at months 1, 3 and 6.
- **Move month-to-month customers to longer contracts** with a small discount.
- **Review pricing for new customers paying over 100 a month.** They are the highest-risk group.
- **Run a weekly watch-list.** Score new customers each week and send the High-risk list to the retention team.
- **Measure and improve.** Track how many contacted customers stay and replace the assumptions with real numbers.

8. Testing evidence

16 automated tests cover data quality, feature engineering, model quality, input validation, the API endpoints and the business calculation. All pass.

```
[ok] test_api.TestAPI.test_health
[ok] test_api.TestAPI.test_home_page_loads
[ok] test_api.TestAPI.test_predict_bad_input_returns_400
[ok] test_api.TestAPI.test_predict_non_json_returns_400
[ok] test_api.TestAPI.test_predict_ok
[ok] test_business.TestBusiness.test_no_flags_means_zero_benefit
[ok] test_business.TestBusiness.test_simple_example
[ok] test_data.TestData.test_add_features_keeps_input_and_handles_zero_tenure
[ok] test_data.TestData.test_data_loads_and_passes_quality_checks
[ok] test_data.TestData.test_validate_detects_missing_values
[ok] test_model.TestModel.test_high_and_low_risk_customers
[ok] test_model.TestModel.test_invalid_inputs_are_rejected
[ok] test_model.TestModel.test_metrics_meet_quality_bar
[ok] test_model.TestModel.test_missing_field_is_rejected
[ok] test_model.TestModel.test_model_file_exists
[ok] test_model.TestModel.test_probability_is_between_0_and_1

Ran 16 tests in 0.127s: OK
```

9. Limitations and next steps

- The dataset is small (500 rows) and simulated (TotalCharges is not exactly tenure times monthly charges), so the near-perfect scores will be lower on real data.
- The test set has only 11 churners; cross-validated AUC (0.993) is the more stable number.
- There is no data on complaints, support calls or service quality.
- Next steps: pilot on live customers, add richer data, host the app online, retrain every quarter.